Vite & Gourmand

Documentation technique

Choix technologiques, architecture, modèle de données, diagrammes, sécurité et déploiement

ECF Développeur Web et Web Mobile — FastDev — Novembre 2026

Dépôt : github.com/Lakagoull/vite-et-gourmand — Application : vite-et-gourmand-grii.onrender.com

Sommaire

1. Réflexions initiales et choix technologiques
2. Architecture de l'application
3. Configuration de l'environnement de travail
4. Modèle de données (relationnel et NoSQL)
5. Diagramme de cas d'utilisation
6. Diagrammes de séquence
7. Règles de gestion
8. Sécurité
9. Accessibilité (RGAA) et RGPD
10. Tests réalisés
11. Déploiement

1. Réflexions initiales et choix technologiques

1.1 Analyse du besoin

Le cahier des charges décrit une application de vente en ligne de menus de traiteur, avec quatre profils (visiteur, utilisateur, employé, administrateur), un catalogue filtrable sans rechargement, un tunnel de commande avec calcul de prix (livraison au kilomètre, remise), un cycle de vie des commandes avec notifications par e-mail, un back-office et des statistiques issues d'une base non relationnelle. Les contraintes transverses sont la **sécurité**, le **RGPD**, l'**accessibilité (RGAA)** et un **déploiement obligatoire**.

Le volume de données est faible (quelques menus, quelques centaines de commandes par an) et l'équipe cliente n'est pas technique : la solution doit être **simple à exploiter**, **peu coûteuse** et **facile à maintenir**.

1.2 Stack retenue et justification

Couche	Choix	Justification	Alternatives étudiées
Front-end	HTML5, CSS, **Bootstrap 5.3**, JavaScript natif	Bootstrap fournit une grille responsive et des composants accessibles (menu mobile, formulaires) ; surchargé par la charte via des variables CSS. JavaScript natif (fetch) suffit pour les filtres dynamiques et le calcul du prix : aucune dépendance, page légère.	React / Vue : surdimensionnés pour quelques interactions, ajoutent une étape de build. Tailwind : demande plus de code de mise en page.
Back-end	**PHP 8** sans framework, architecture MVC légère, **PDO**	Stack proposée par le sujet, hébergeable partout. L'absence de framework permet de démontrer la maîtrise des mécanismes (routage, sessions, CSRF, requêtes préparées) et garde un code court et lisible.	Symfony / Laravel : très complets mais masquent ces mécanismes ; Node.js (Express) : possible, mais PHP est demandé par l'exemple de stack.
Base relationnelle	**MySQL 8** (MariaDB en local)	Modèle très relationnel (menus, plats, allergènes en n-n ; commandes ; avis) : intégrité référentielle, transactions et verrous indispensables pour le stock.	PostgreSQL : équivalent ; MySQL retenu car présent dans XAMPP et proposé gratuitement par l'hébergeur choisi.
Base NoSQL	**MongoDB**	Exigée pour les statistiques. Une collection dénormalisée (une copie par commande) se prête bien aux agrégations (`$group` , `$sum`) par menu et par période.	Redis : peu adapté aux agrégations ; Elasticsearch : trop lourd.
E-mails	**PHPMailer** (SMTP)	Bibliothèque de référence : encodage UTF-8, protection contre l'injection d'en-têtes, SMTP chiffré.	Fonction `mail()` : peu fiable et non sécurisée.
Géolocalisation	**API Adresse** (Base Adresse Nationale)	Service public, gratuit, sans clé : vérifie l'adresse, identifie la commune (code INSEE de Bordeaux) et donne les coordonnées GPS pour le calcul des kilomètres.	Google Maps : clé et facturation obligatoires.
Déploiement	**Docker** sur **Render** + **Aiven** (MySQL) + **MongoDB Atlas**	Offres gratuites sans carte bancaire, HTTPS automatique, déploiement continu depuis GitHub, serveurs dans l'UE. Docker garantit un environnement identique à chaque déploiement.	Heroku : plus d'offre gratuite ; Fly.io : carte bancaire requise ; Vercel : pas adapté à PHP avec extension MongoDB.

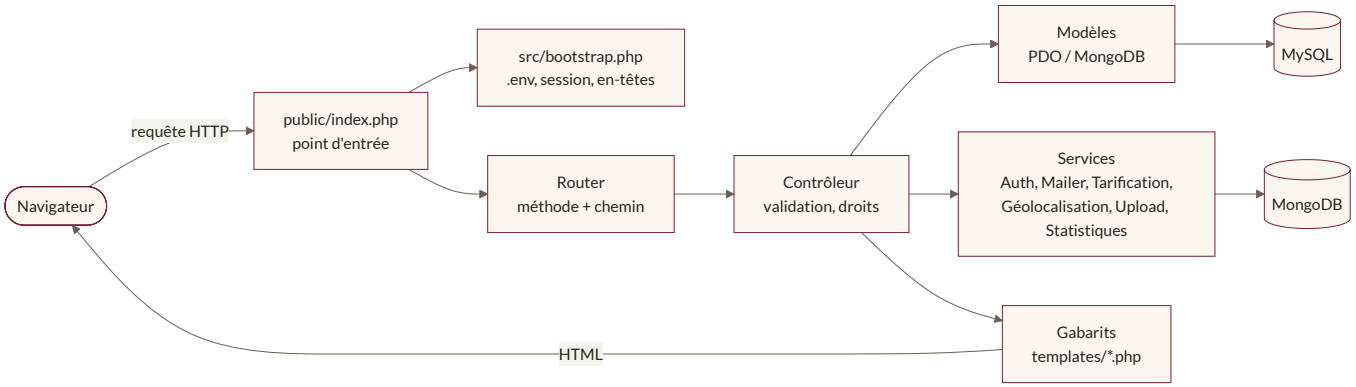
1.3 Dépendances

Volontairement réduites au minimum (surface d'attaque et maintenance) : `phpmailer/phpmailer` (via Composer), Bootstrap 5.3.3 et les polices Lato / Playfair Display (**hébergés localement**, voir RGPD). Les extensions PHP utilisées sont `pdo_mysql`, `mongodb`, `gd`, `mbstring` et `openssl`.

2. Architecture de l'application

2.1 Organisation MVC

L'application suit le modèle **MVC** avec un **point d'entrée unique** (*front controller*) : toutes les requêtes passent par `public/index.php`, qui charge la configuration, démarre une session sécurisée, envoie les en-têtes de sécurité puis confie la requête au routeur.



2.2 Arborescence

```
vite-et-gourmand/
├── public/                 Racine web (seul dossier exposé)
│   ├── index.php           Point d'entrée unique + déclaration des routes
│   ├── .htaccess           Réécriture d'URL Apache
│   └── assets/             CSS, JS, polices, images, Bootstrap
├── src/
│   ├── bootstrap.php       Configuration, session, en-têtes de sécurité
│   ├── helpers.php         Échappement, CSRF, rendu, messages flash
│   ├── Database.php        Connexion PDO unique (TLS en production)
│   ├── Router.php          Routeur minimaliste
│   ├── Controllers/        Page, Menu, Contact, Auth, Commande, Espace,
│   │                       Gestion, GestionCommande, GestionMenu, Admin
│   ├── Models/             Menu, Plat, Commande, Utilisateur, Avis, Horaire
│   └── Services/           Auth, Mailer, Tarification, Geolocalisation,
│                           Upload, Statistiques
├── templates/              layout.php, pages/, partials/
├── database/               schema.sql, data.sql, mongo-init.js,
│                           sync-mongo.php, import.php
├── docs/                   PDF (charte, manuel, documentations)
├── Dockerfile, docker/     Image de production
└── render.yaml             Déploiement (Infrastructure as Code)
```

2.3 Principes de conception

- **Séparation des responsabilités** : les contrôleurs valident les entrées et vérifient les droits ; les modèles contiennent tout le SQL ; les services portent la logique réutilisable (calcul du prix, envoi d'e-mails…) ; les gabarits ne font qu'afficher.
- **Une seule source de vérité pour les prix** : le service `Tarification` est utilisé à la fois par l'API de devis (affichage en direct) et à la validation de la commande. Le prix affiché en JavaScript n'est jamais repris tel quel.
- **Amélioration progressive** : la liste des menus est rendue côté serveur et fonctionne sans JavaScript ; le JavaScript ajoute le filtrage instantané via l'API `/api/menus`.
- **Configuration par variables d'environnement** (fichier `.env` en local, tableau de bord de l'hébergeur en production) : aucun secret dans le code.

3. Configuration de l'environnement de travail

Outil	Version	Rôle
Windows 11	—	Système du poste de développement
Visual Studio Code	—	Éditeur de code
XAMPP	PHP 8.0.30, MariaDB 10.4	PHP en ligne de commande et base relationnelle locale
Extension PHP mongodb	1.20.1	Pilote MongoDB pour PHP (ajoutée dans `php.ini`, avec `gd`)
MongoDB Server + mongosh	9.0 / 2.14	Base NoSQL locale et son terminal
Composer	2.11	Gestion des dépendances PHP et autoload PSR-4
Git + GitHub CLI	2.54 / 2.103	Versionnement, dépôt public, branches
Microsoft Edge	—	Tests, outils de développement, export PDF

Mise en route en local

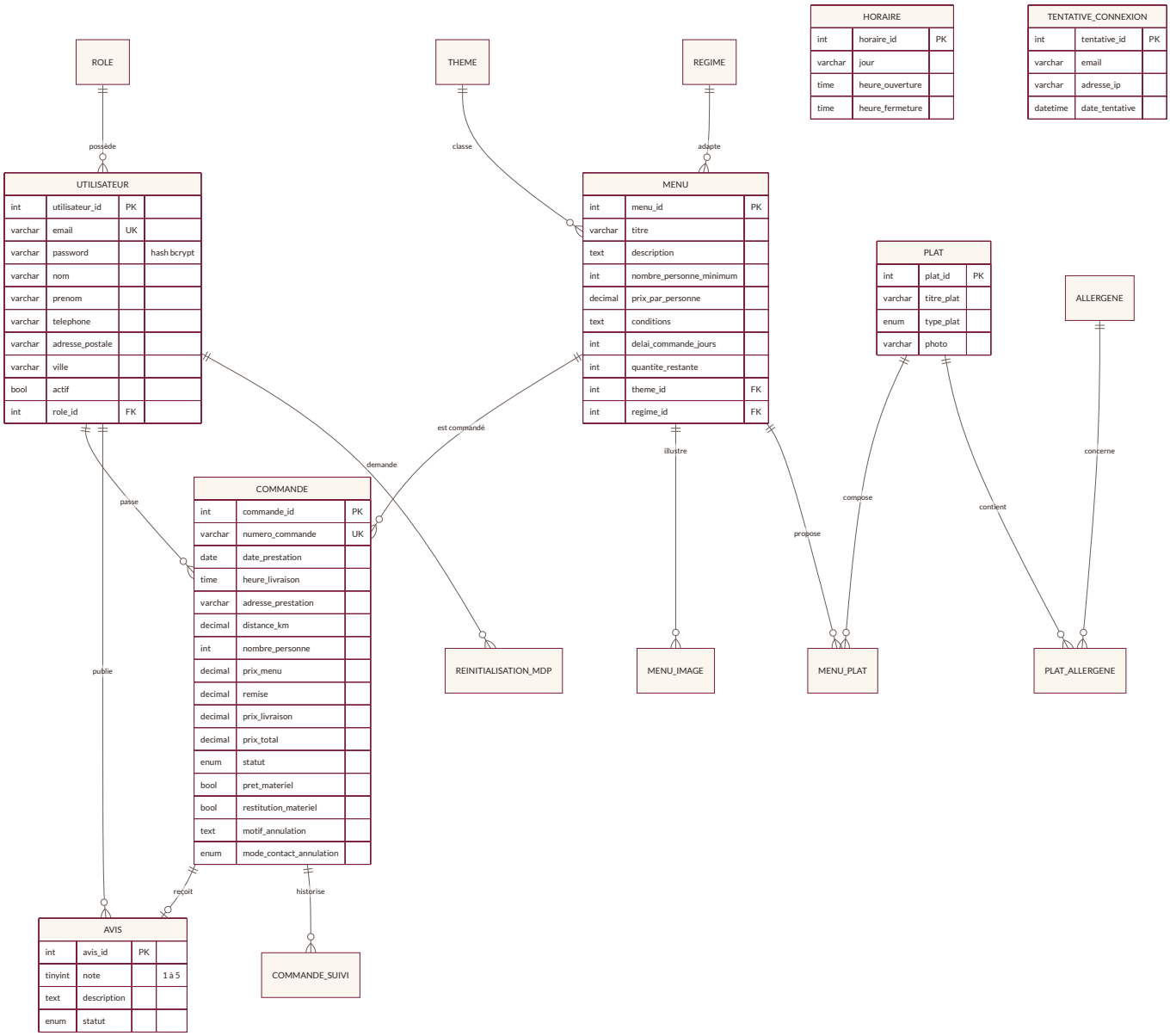
```
git clone https://github.com/Lakagoull/vite-et-gourmand.git
cd vite-et-gourmand
composer install
cp .env.example .env                       # adapter les accès
mysql -u root -p < database/schema.sql     # ou : php database/import.php
mysql -u root -p < database/data.sql
mongosh "mongodb://127.0.0.1:27017" database/mongo-init.js
php database/sync-mongo.php                # copie des commandes vers MongoDB
php -S localhost:8000 -t public public/index.php
```

En local, si `SMTP_HOST` est vide, les e-mails sont écrits dans `storage/mails/` (fichiers `.eml`), ce qui permet de tester tous les envois sans serveur de messagerie.

4. Modèle de données

4.1 Modèle relationnel (MySQL)

Le modèle reprend le MCD de l'annexe 1 et le complète pour couvrir tout le cahier des charges.



> Tables simples non détaillées ci-dessus : `role`, `theme`, `regime`, `allergene` (identifiant + libellé), `menu_image` (chemin, texte alternatif, ordre), `commande_suivi` (statut, date), `reinitialisation_mdp` (empreinte du jeton, expiration, utilisé). `tentative_connexion` et `horaire` sont indépendantes.

4.2 Écarts avec l'annexe et justifications

Évolution	Justification
Prix en `DECIMAL` au lieu de `DOUBLE`	Les nombres flottants provoquent des erreurs d'arrondi sur les montants (0,1 + 0,2 ≠ 0,3).
Prix **figés** dans la commande (menu, remise, livraison, total, distance)	Un changement de tarif ultérieur ne modifie pas l'historique ni le chiffre d'affaires.

Évolution	Justification
Table `menu_image`	Le menu dispose d'une *galerie* d'images (relation 1-n), avec un texte alternatif obligatoire (RGAA).
Table `commande_suivi`	Le suivi doit lister tous les états avec date et heure.
Colonne `delai_commande_jours`	Rend vérifiable automatiquement la condition « à commander x jours avant ».
Colonne `actif`	Rendre inutilisable un compte employé sans le supprimer (historique conservé).
Tables `reinitialisation_mdp` et `tentative_connexion`	Mot de passe oublié sécurisé et protection contre la force brute.
Photo du plat en chemin de fichier (`VARCHAR`) au lieu de `BLOB`	Base plus légère, images servies directement et mises en cache par le serveur web.
Contraintes `CHECK` , clés uniques, `ON DELETE CASCADE` ciblés	Intégrité garantie par la base (note 1-5, stock ≥ 0, un seul avis par commande…).

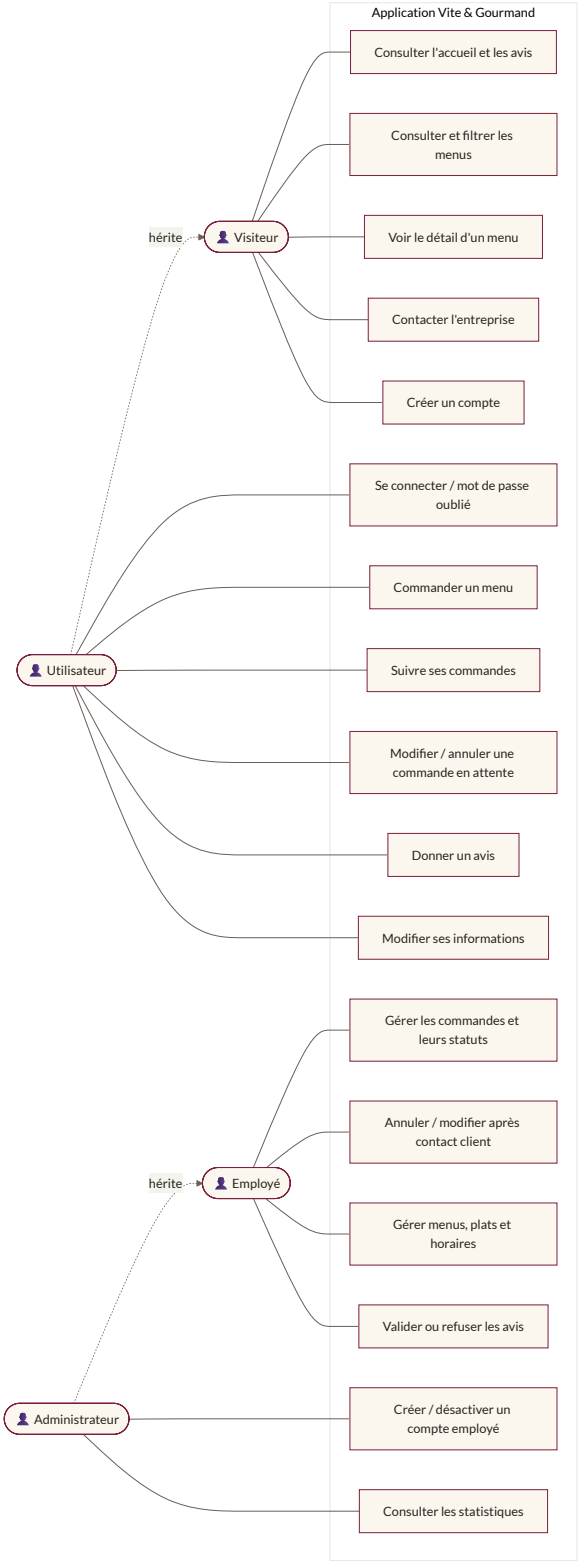
4.3 Base NoSQL (MongoDB)

Collection `statistiques_commandes` : un document par commande, copie dénormalisée créée à la commande et mise à jour à chaque changement (statut, modification). Un schéma de validation (`$jsonSchema`) et deux index (`commande_id` unique ; `menu_id + date_commande` pour les filtres) sont créés par `mongo-init.js` .

```
{
  "commande_id": 2, "numero_commande": "VG-2026-0002",
  "menu_id": 2, "menu_titre": "Menu de Pâques Printanier",
  "nombre_personne": 9, "prix_total": 302.21,
  "date_commande": ISODate("2026-10-01T08:05:00Z"), "statut": "terminee"
}
```

Le chiffre d'affaires par menu est calculé par un pipeline d'agrégation : `$match` (statut ≠ annulée, menu, période) → `$group` par `menu_id` (`$sum` des commandes et des montants) → `$sort` . Une erreur MongoDB n'empêche jamais l'enregistrement d'une commande (elle est journalisée) ; le script `sync-mongo.php` permet de reconstruire la collection à partir de MySQL.

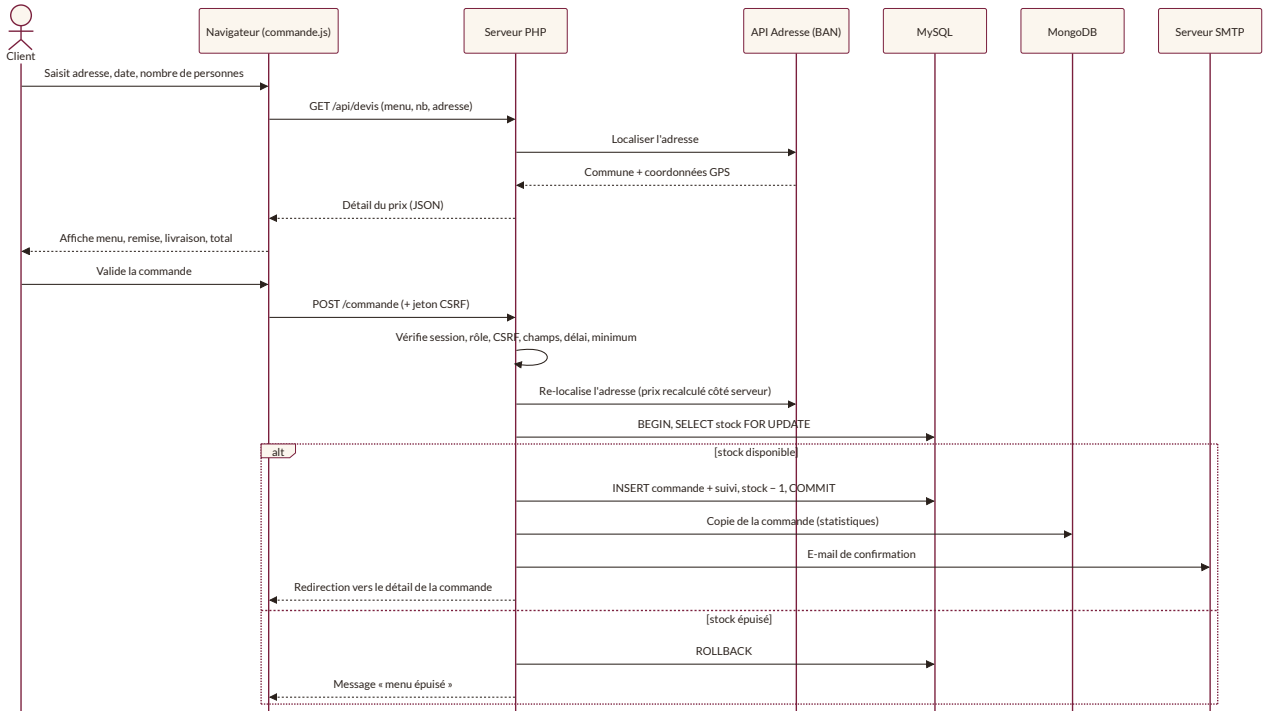
5. Diagramme de cas d'utilisation



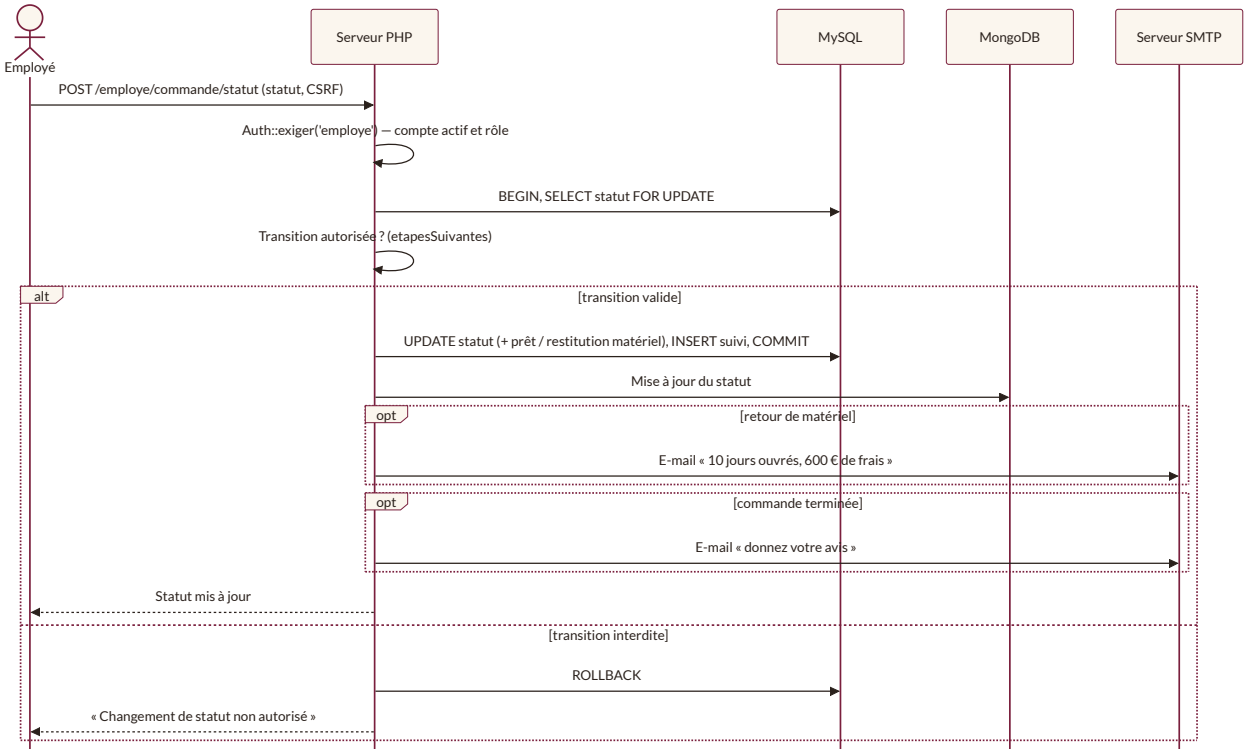
L'utilisateur dispose de toutes les fonctions du visiteur ; l'administrateur de toutes celles de l'employé (vérifié dans le service `Auth::exiger()`). La connexion est commune aux trois profils authentifiés.

6. Diagrammes de séquence

6.1 Commande d'un menu



6.2 Changement de statut par un employé



7. Règles de gestion

Règle	Mise en œuvre
Prix du menu	prix par personne × nombre de personnes ; le prix affiché sur le catalogue correspond au nombre minimum de personnes.
Remise de 10 %	Appliquée si nombre de personnes ≥ minimum du menu + 5.
Livraison	Gratuite si la commune est Bordeaux (code INSEE 33063) ; sinon 5 € + 0,59 € × km. Distance calculée à vol d'oiseau depuis l'atelier (formule de haversine).
Minimum de personnes	Refus en dessous du minimum du menu (contrôle serveur + attribut `min`).
Délai de commande	Date de prestation ≥ aujourd'hui + délai du menu (au moins 1 jour), au plus 1 an.
Stock	Décrémenté à la commande dans une transaction avec verrou ; rendu en cas d'annulation ; menu non commandable à 0.
Modification / annulation client	Uniquement en statut « en attente » ; le menu ne peut pas être changé.
Modification / annulation employé	Mode de contact (téléphone / e-mail) et motif obligatoires ; impossible une fois la livraison commencée.
Cycle de vie	en attente → acceptée → en préparation → en cours de livraison → livrée → (attente retour matériel) → terminée ; aucune étape ne peut être sautée.
Avis	Un seul par commande terminée, note 1 à 5, publié après validation par un employé.
Comptes	Inscription = rôle utilisateur imposé ; comptes employés créés par l'administrateur ; aucun compte administrateur créable depuis l'application.
Suppression d'un menu	Refusée s'il a déjà été commandé (historique et chiffre d'affaires).

8. Sécurité

La sécurité a été traitée à chaque fonctionnalité, puis vérifiée par des tests d'attaque (voir chapitre 10).

Risque	Protection mise en place
Injection SQL	100 % des requêtes avec paramètres passent par des **requêtes préparées PDO**, émulation désactivée (`ATTR_EMULATE_PREPARES = false`). Les filtres sont validés et typés (`filter_var`).
XSS (injection de script)	Toute donnée affichée passe par la fonction `e()` (`htmlspecialchars`). Côté JavaScript, construction du DOM avec `textContent` (jamais `innerHTML` avec des données). **Content-Security-Policy** stricte : aucun script ni style en ligne autorisé, ressources du seul domaine.
CSRF	Jeton aléatoire de 256 bits par session, vérifié avec `hash_equals` sur chaque formulaire POST ; cookie `SameSite=Lax` .
Mots de passe	Hachage **bcrypt** (`password_hash`), re-hachage automatique si l'algorithme évolue ; politique : 10 caractères, majuscule, minuscule, chiffre, caractère spécial.
Force brute	Blocage après 5 échecs par e-mail (et 20 par IP) sur 15 minutes ; temps de réponse constant même pour un compte inexistant ; messages d'erreur identiques (pas d'énumération des comptes).
Vol de session	Cookie `HttpOnly` , `Secure` en HTTPS, `SameSite` ; `session_regenerate_id` à la connexion ; mode strict ; destruction complète à la déconnexion.
Mot de passe oublié	Jeton aléatoire de 256 bits, seule son empreinte SHA-256 est stockée ; valable 1 h, usage unique, anciens liens invalidés ; réponse identique que l'adresse existe ou non.
Contrôle d'accès	`Auth::exiger()` sur chaque action protégée : rôle vérifié et compte toujours actif en base (un employé désactivé est déconnecté immédiatement). Un client ne peut accéder qu'à ses propres commandes (contrôle du propriétaire, sinon 404).
Élévation de privilèges	Le rôle est toujours imposé côté serveur (inscription = utilisateur, création par l'admin = employé) ; les champs ajoutés dans la requête sont ignorés.
Redirection ouverte	Le paramètre de retour après connexion n'accepte que des chemins internes (pas de `//` , pas de schéma, pas d'antislash).
Envoi de fichiers	Taille limitée (3 Mo), **vrai type vérifié** (`finfo`), image **ré-encodée avec GD** (suppression des métadonnées et de tout contenu caché), nom aléatoire, extension imposée `.jpg` .
Manipulation des prix	Prix, remise et livraison recalculés côté serveur à partir de la base ; le stock est vérifié sous verrou (pas de survente).
Spam	Champ piège invisible et limitation d'un envoi par minute (contact, mot de passe oublié).
En-têtes HTTP	`Content-Security-Policy` , `Strict-Transport-Security` , `X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Permissions-Policy` ; signature serveur masquée.
Secrets et erreurs	Accès aux bases dans des variables d'environnement (jamais dans le dépôt, `.env` ignoré par Git) ; erreurs détaillées affichées uniquement en développement ; connexion MySQL chiffrée (TLS) avec vérification du certificat en production.

9. Accessibilité (RGAA) et RGPD

9.1 Accessibilité

- Contrastes calculés et conformes (≥ 4,5:1) pour toutes les associations texte / fond (voir charte graphique).
- Structure sémantique : `header`, `nav`, `main`, `footer`, titres hiérarchisés, fil d'Ariane, `aria-current` sur la page active.
- Lien d'évitement « Aller au contenu principal » ; navigation complète au clavier ; focus visible renforcé.
- Formulaires : chaque champ a une étiquette ; erreurs listées en haut avec liens vers les champs et reliées au champ (`aria-invalid`, `aria-describedby`) ; formats attendus indiqués.
- Information jamais portée par la seule couleur : conditions du menu avec icône et titre, statuts avec libellé.
- Zones dynamiques annoncées (`aria-live`) : nombre de résultats des filtres, détail du prix.
- Images : texte alternatif obligatoire pour la galerie ; images décoratives ignorées (`alt=""`).
- Graphiques : description textuelle, barres atteignables au clavier, tableau de données équivalent, adaptation au mode contraste élevé.
- Respect de la préférence « réduire les animations » ; zones cliquables d'au moins 44 px.

9.2 RGPD

- Minimisation : seules les données nécessaires à la commande et à la livraison sont demandées.
- Consentement explicite aux CGV et à la politique de confidentialité à l'inscription.
- Pages mentions légales, CGV et politique de confidentialité (finalités, durées de conservation, droits, CNIL).
- Droit de rectification depuis l'espace client.
- Aucun cookie publicitaire ni de mesure d'audience : uniquement le cookie de session (exempté de consentement).
- Bootstrap et polices hébergés sur le serveur : aucune adresse IP de visiteur transmise à un CDN ou à Google Fonts.
- Hébergement de l'application dans l'Union européenne (région Francfort).

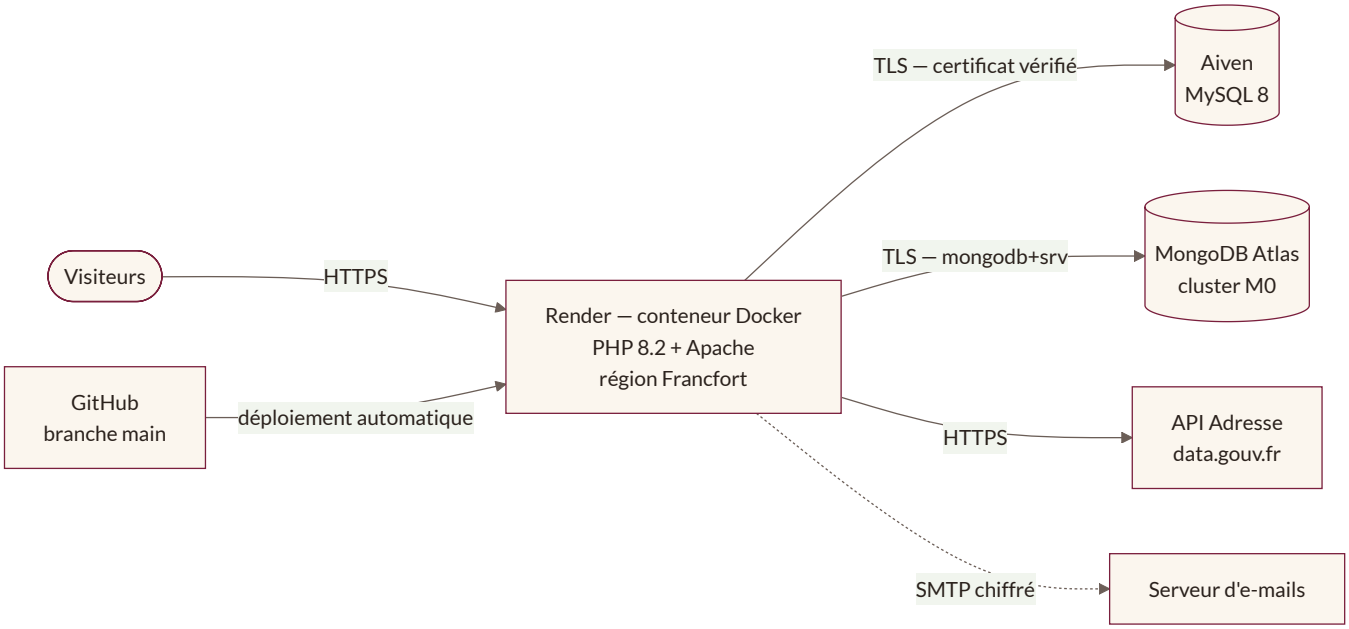
10. Tests réalisés

Chaque fonctionnalité a été testée sur une branche dédiée avant sa fusion dans `develop`, au moyen de scénarios automatisés en ligne de commande (requêtes HTTP réelles avec gestion des cookies et des jetons CSRF) et de vérifications visuelles (captures d'écran bureau et mobile).

Domaine	Scénarios vérifiés	Résultat
Catalogue	Chaque filtre seul et combinés ; paramètre malveillant (`theme=abc'OR1=1`) ignoré	**Conforme**
Contact	Envoi sans jeton CSRF (refus 419), champs invalides, balise `<script>` neutralisée	**Conforme**
Comptes	Mot de passe faible refusé, inscription + mail de bienvenue, rôles et accès (403), redirections malveillantes, blocage après 5 échecs, réinitialisation (lien unique, expiré, réutilisé)	**Conforme**
Commande	Prix (Bordeaux, Mérignac, Arcachon, remise), minimum, délai, stock décrémenté puis rendu, copie MongoDB, accès à la commande d'un autre client (404), modification après acceptation refusée	**Conforme**
Avis	Refusé avant la fin de la commande, note hors limites, second avis, validation / refus	**Conforme**
Employé	Cycle de vie complet, saut d'étape refusé, prêt de matériel, mails automatiques, annulation sans contact refusée	**Conforme**
Envoi d'images	Image valide acceptée ; script PHP renommé en `.jpg` refusé	**Conforme**
Administrateur	Création d'employé (rôle forcé, mail sans mot de passe), désactivation avec session ouverte, statistiques MongoDB comparées à un calcul MySQL (tous filtres)	**Conforme**
Production	Pages, connexion, base MySQL et statistiques MongoDB sur le site en ligne, en-têtes de sécurité	**Conforme**

11. Déploiement

11.1 Architecture de production



11.2 Démarche et étapes

1. **Conteneurisation** : le `Dockerfile` part de l'image officielle `php:8.2-apache`, installe les extensions `pdo_mysql`, `gd` et `mongodb`, place la racine web sur `public/`, active la réécriture d'URL, passe PHP en configuration de production et installe les dépendances Composer sans les outils de développement. Le script `docker/entrypoint.sh` adapte Apache au port fourni par l'hébergeur et écrit le certificat MySQL transmis en variable d'environnement.
2. **Base relationnelle** : création d'un service MySQL gratuit sur Aiven ; récupération de l'hôte, du port, de l'utilisateur, du mot de passe et du certificat d'autorité. Import de `schema.sql` et `data.sql` avec `php database/import.php production.env` (connexion TLS ; le client MySQL de XAMPP ne gérant pas l'authentification `caching_sha2_password` de MySQL 8, l'import passe par PDO).
3. **Base NoSQL** : création d'un cluster gratuit M0 sur MongoDB Atlas, d'un utilisateur de base, et autorisation réseau (liste d'accès IP) ; exécution de `mongo-init.js` (collection, validation, index) puis `php database/sync-mongo.php` pour copier les commandes depuis MySQL.
4. **Hébergement** : sur Render, création d'un *Blueprint* à partir du fichier `render.yaml` du dépôt (service web Docker, offre gratuite, région UE, branche `main`). Les secrets (accès aux bases, certificat, SMTP) sont saisis dans le tableau de bord, jamais versionnés.
5. **Mise en production** : fusion de `develop` dans `main` ; Render construit l'image et la déploie automatiquement. Chaque nouvelle fusion dans `main` redéploie l'application.
6. **Vérification** : test de toutes les pages, de la connexion, des données MySQL, des statistiques MongoDB et des en-têtes de sécurité sur l'URL publique.

11.3 Variables d'environnement

Variable	Rôle
`APP_ENV`, `APP_URL`	Mode (prod : erreurs masquées) et adresse publique (liens des e-mails)

Variable	Rôle
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`	Accès MySQL
`DB_CA_CERT`	Certificat d'autorité pour la connexion TLS vérifiée
`MONGO_URI`, `MONGO_DB`	Accès MongoDB Atlas
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `MAIL_FROM`, `MAIL_ENTREPRISE`	Envoi des e-mails et adresse de l'entreprise (formulaire de contact)
`TRUST_PROXY`	Lecture de l'IP réelle derrière le proxy de l'hébergeur (limiteur de tentatives)

11.4 Limites connues et évolutions

- **Mise en veille** de l'offre gratuite après 15 minutes d'inactivité (premier chargement ≈ 30 s). Évolution : offre payante « always on ».
- **Stockage éphémère** : les photos envoyées en production sont perdues à chaque redéploiement. Évolution : stockage objet externe (type S3) ou disque persistant.
- **E-mails** : l'envoi réel nécessite de renseigner un serveur SMTP (par exemple Brevo) dans les variables d'environnement ; sans SMTP, les e-mails sont enregistrés côté serveur.
- **Distance** calculée à vol d'oiseau : un calcul d'itinéraire routier nécessiterait un service de routage.